Digital Signal Processing

A Unified Course Companion

Compiled from lecture notes
ISI Kolkata — BSDS

Notes dated 24 July – 28 August 2026

Preface: How This Book Is Organized

This volume consolidates five sets of lecture notes from the introductory Digital Signal Processing (DSP) course into a single, continuous narrative. The five source documents were delivered over five weeks and, read in isolation, each stands as a self-contained handout; read together, they form one continuous argument that runs from "what is a signal?" all the way to the convolution and correlation sums that make LTI systems tractable. The purpose of this compilation is to make that continuous argument visible.

Three editorial principles were used throughout:

1. **No content was dropped.** Every definition, key result, worked example, and "to derive / write up" exercise from the original five note sets appears somewhere in this book, usually verbatim in mathematical content, lightly re-flowed in prose.
2. **Overlaps were merged, not duplicated.** Several ideas are introduced more than once across the original notes as the course circles back to them with more machinery (the impulse-train representation of a signal, for instance, appears in the notes for Lecture 2, Lecture 3, Lecture 4, and Lecture 5, each time doing a little more work). Rather than repeating the same box four times, this book states the idea fully once, and every later occurrence is a short *Connections* box that explains what new work is being done with the already-established fact.
3. **Every chapter opens with a roadmap and closes with a bridge.** The roadmap says where the chapter sits in the course outline (Ch. 1) and what it assumes; the closing section previews what the next chapter will build on top of it.

Colour key used throughout:

Definition

— a term is being fixed precisely for the first time.

Key Result

— a formula or theorem worth memorizing.

Worked Example

— a fully solved problem.

Connections

— an explicit pointer to where this idea came from or where it is going.

To Derive / Work Out

— left for hand-worked practice, as in the originals.

Contents

Course Map

The original Lecture 1 notes laid out a running syllabus for the course (roughly 13–15 hours). Figure 1 shows how the five lecture-note sets compiled into this book map onto that syllabus, and how each chapter's central result feeds the next chapter's starting point.

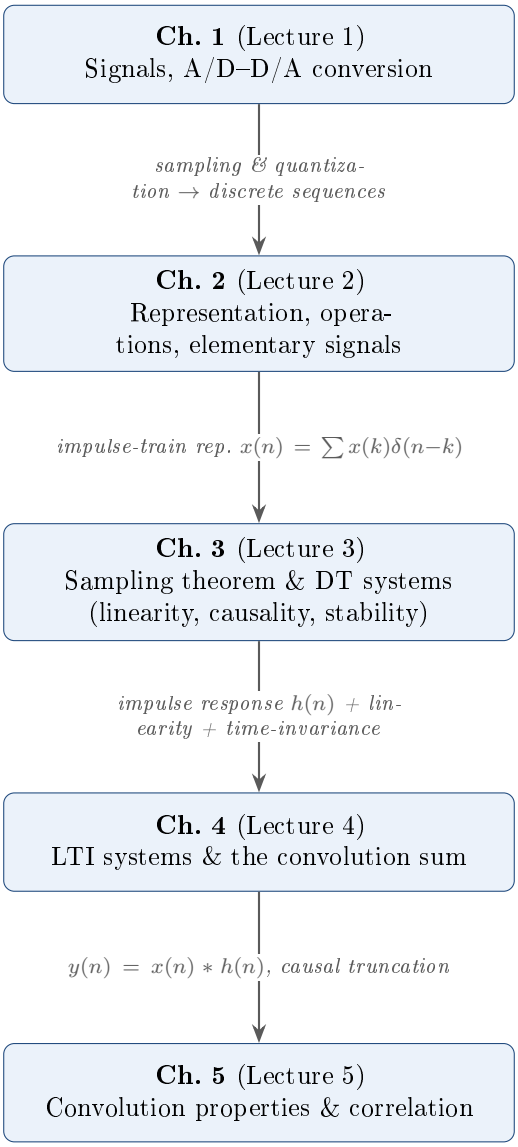


Figure 1: How the five lecture-note sets chain into one argument.

Chapter 1

Introduction: Signals, Their Processing, and A/D–D/A Conversion

Where we are. This is the first chapter of the course, so it assumes only "fair knowledge of basic mathematics." **What it does.** It answers three questions in order: what *is* a signal, why must it be *processed*, and what happens physically when an analog signal is turned into numbers a computer can store (and back again). **Where it leads.** The four-stage pipeline built at the end of this chapter — sampling, holding, quantizing, encoding — is revisited with full mathematical machinery in Ch. 3 (the sampling theorem and aliasing) and is the physical justification for why Ch. 2 onward works entirely with *discrete* sequences $x(n)$ instead of continuous functions $x(t)$.

1.1 Outline of the Course

The course (roughly 13–15 hours) presumes familiarity with basic mathematics and proceeds through the following topics, each of which is a running table of contents to check off against as the book progresses:

- Signals and systems: introduction/review
- Continuous-time (CT) signals; discrete-time (DT) signals and systems in the time domain
- Typical processing of DT signals with DT systems
- LTI systems: characterization and classification (Chs. 3 and 4)
- Sampling in time, aliasing, interpolation, quantization (§ 1.8 and Ch. 3)
- Basic operations like convolution (Chs. 4 and 5)
- DT signals in the frequency domain; DFT, DTFT; elementary Z-transform (beyond this volume's five source lectures)
- Transfer function, frequency response; simple digital filter design
- Realization of digital filters: FIR and IIR
- Correlation in time and frequency domain (Ch. 5)

To Derive / Work Out

As the course proceeds, map each bullet above to the chapter/section where it is formally treated — this is the master checklist for the whole book.

1.2 Introduction to Signal and Wave

- A **wave** is a movement in time in a medium. A wave needs a medium.
- A **signal** contains information, and its variations are represented in the form of waves.
- Signal information is about the state or behavior of a physical system.

Definition: Signal

A signal is a variation of the behavior of the state of a physical system:

$$\text{Signal} \Rightarrow Y = f(x), \qquad x = \text{time, temperature, density, pressure, etc.}$$

Examples: variations of acoustic pressure in a speech signal; variations of reflectance value in a picture/image; variations with depth of density, porosity, and electrical resistivity in geophysics; variations of air pressure, temperature, and wind speed with altitude in meteorology; variations of average budget, crime rate, or pounds of fish caught in demographic studies.

To Derive / Work Out

Note the common thread across all these examples: in each case, identify what plays the role of the independent variable x and what plays the role of the amplitude Y.

1.3 Types of Signal

Signals may be classified along several independent axes: natural vs. synthetic, by number of dimensions, by source, continuous vs. discrete, and analog vs. digital.

A subtlety worth remembering

A signal is conventionally described as a function of time, but in DSP this loses its significance — the independent variable need not be time at all (recall the geophysics and image examples above). Moreover, **all CT (continuous-time) signals are analog, but not the other way around** — "continuous-time" and "analog" are not synonyms. The precise 2×2 categorization (continuous/discrete in time crossed with continuous/discrete in amplitude) is given in § 1.7.

1.4 Why Signal and Its Processing?

Why Signal? A signal contains information. **Why Processing?** Having information is not sufficient — it needs to be processed to be made useful (extracted, cleaned, transformed). Typical operations include modulation/demodulation, multiplexing/demultiplexing, and filtering.

Corruption — a motivating example. Two classic corrupting sources: 50 Hz power-line pickup (induction), and electromyography (EMG) signals corrupting an electrocardiogram (ECG) recording. The ECG waveform (P–QRS–T complex, with PR interval, PR segment, ST segment, and QT interval landmarks) illustrates a clean trace versus one with visible EMG-type noise superimposed.

To Derive / Work Out

Sketch the P–QRS–T complex from memory and label the PR interval, PR segment, QRS complex, ST segment, and QT interval. Then explain why 50 Hz pickup and EMG artifacts are particularly problematic for ECG interpretation (frequency overlap with the signal band).

1.5 Ways to Perform Signal Processing

Signal processing can be carried out in three ways: **analog**, **digital**, or **mixed**.

Open question raised in class

Why not only analog, or only digital?

To Derive / Work Out

Draft your own answer: think about precision/repeatability/storage/programmability (favoring digital) versus bandwidth, power, and the unavoidable need for analog interfacing at the sensor and actuator (favoring analog or mixed-signal systems). Section 2.1 in the next chapter returns to this question with a concrete list of "why DSP" and "why not DSP" arguments.

1.6 Mathematical Representation of a Signal

Definition

A signal is a function f that maps an independent variable to a value (amplitude):

$$x = f(t).$$

Examples, mathematically:

$$\text{Linear:}\quad x(t) = 2t + 4, \qquad \text{Quadratic:}\quad x(t) = t^2 + 1,$$
$$\text{Exponential:}\quad x(t) = e^t, \qquad \text{Sinusoidal:}\quad x(t) = A\sin(\omega t + \phi).$$

More generally, a multi-component signal (as in speech) can be written as a sum of amplitude- and frequency-modulated sinusoids:

$$\sum_{i=1}^{N} A_i(t)\,\sin\big(2\pi F_i(t)\,t + \theta_i(t)\big).$$

Worked Example: superposition

$s_1(t) = 2t$, $s_2(t) = \frac{1}{2}t^2 \;\Rightarrow\; s_1(t) + s_2(t) = 2t + \frac{1}{2}t^2$.

A two-variable (image-like) signal example: $S(x, y) = 3x + 10xy + 2x^2y$.

To Derive / Work Out

- *For the speech sum $\sum_i A_i(t)\sin[2\pi F_i(t)t + \theta_i(t)]$: note that A_i, F_i, θ_i are themselves functions of time (time-varying amplitude, instantaneous frequency, phase) — the seed of later time–frequency analysis (short-time Fourier transform).*
- *Verify the superposition example by direct substitution, and think about why linearity of "+" here foreshadows the definition of a linear* system *(formalized in § 3.5).*
- *For $S(x, y) = 3x+10xy+2x^2y$: identify which terms are linear in x, linear in y, and which are genuinely bivariate (coupling) terms — the two-dimensional analogue of a 1-D signal, relevant for images.*

1.7 Signal Categorization

Two independent axes classify a signal: the **time axis** (continuous-time vs. discrete-time) and the **amplitude axis** (continuous-amplitude vs. discrete-amplitude). Crossing them gives four categories.

	Continuous-Time (defined for all real t)	**Discrete-Time** (defined only at instants nT)
Continuous-Amplitude	(1) *Analog signal*: continuous in time and amplitude. E.g. audio sound, human voice, temperature.	(3) *Sampled* (discrete-time analog) signal: discrete in time, continuous in amplitude. E.g. output of sampling an analog signal, $x[n]$.
Discrete-Amplitude	(2) Continuous-time, discrete-amplitude signal: continuous in time, but only specific (discrete) levels. E.g. ideal rectangular wave, on–off signal, digital level control.	(4) *Digital signal*: discrete in time and amplitude (typically binary 0/1). E.g. binary data, computer data, digital communication signals.

Table 1.1: The 2×2 categorization of signals.

Key Result: Analog–Digital pipeline

$$\text{Analog signal} \xrightarrow{\text{Sampling}} \text{Sampled signal} \xrightarrow{\text{Quantization}} \text{Quantized signal} \xrightarrow{\text{Encoding}} \text{Digital signal},$$

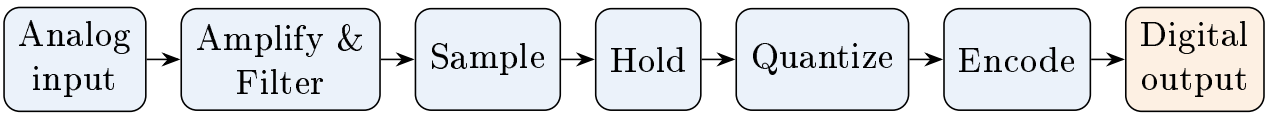
with reconstruction/decoding running the pipeline in reverse.

To Derive / Work Out

Real-world examples to keep straight: continuous-amplitude signals include sound waves, human voice, temperature; discrete-amplitude signals include computer data, mobile data, digital communication. Draw the 2×2 grid yourself with one sketch per cell (a continuous curve $x(t)$, a continuous-time square wave, a stem plot $x[n]$, and a binary-level stem plot) so the distinction between "discrete in time" and "discrete in amplitude" is internalized.

1.8 Analog ⇒ Digital ⇒ Analog

1.8.1 Analog-to-Digital Converter (ADC) Pipeline



The signal's state moves through the pipeline as follows:

- continuous-in-time/continuous-in-amplitude →
- discrete-in-time/continuous-in-amplitude (after *Sample*) →
- discrete-in-time/continuous-in-amplitude (after *Hold*) →
- discrete-in-time/discrete-in-amplitude (after *Quantization* and *Encoding*).

An end-to-end DSP system (e.g. a PIC32-type microprocessor) looks like:

Analog Input → Anti-aliasing Filter → Sample and Hold → ADC → $\underbrace{\text{DSP Algorithm (software)}}_{\text{inside the microprocessor}}$ → DAC

with the sampling step written $x_a(t) \to x_a(nT) \Rightarrow x(n)$.

1.8.2 Sample and Hold (S/H)

The pipeline of representations in Sample-and-Hold is:

1. Continuous-time analog input $x_a(t)$.
2. Sampling instants (every T_s seconds): $t = 0, T_s, 2T_s, 3T_s, \ldots$
3. Sample-and-hold output $x_{sh}(t)$ (a staircase waveform): sample the value, then hold it constant until the next sampling instant.
4. The three signals — analog input $x_a(t)$, sampled values $x[n] = x_a(nT_s)$, and the S/H output $x_{sh}(t)$ — are shown together on one timing diagram.
5. A timing diagram shows the sampling clock (high = sample, low = hold), the S/H switch (closed = sample, open = hold), and the S/H output (holds the applied value for the ADC).

Margin annotation: illustrative hold-time values mentioned in the notes: 10 ms, 15 ms, 20 ms.

Key Result: Nyquist-type inequality (preview)

$$f_s \gtrless 2f_0$$

relating sampling frequency f_s to signal frequency f_0 — flagged here for formal treatment under sampling and aliasing.

⇄ Connections

This inequality is only a sketch. Section 3.2 states and uses the precise Nyquist–Shannon sampling theorem ($f_s > 2f_{\max}$), defines the Nyquist rate and Nyquist frequency, and works a full three-tone aliasing example. Everything in this box is superseded — but motivated — by that section.

To Derive / Work Out

When the course formally covers sampling and aliasing (§ 3.2), come back and confirm: the precise statement $f_s \geq 2f_{\max}$ (Nyquist rate), why the boundary case $f_s = 2f_{\max}$ is delicate, and what happens (aliasing) when $f_s < 2f_{\max}$.

1.8.3 Quantization and Encoding

Pipeline: continuous-time analog signal $x_a(t)$ → sampled signal $x[n] = x_a(nT_s)$ → uniform quantization → quantized signal $x_q[n]$ → encoding (binary representation) → encoded digital signal $b[n]$.

Definition: Uniform quantization

The amplitude range $[-A_{\max}, A_{\max}]$ is divided into $L = 2^B$ discrete quantization levels, where B is the number of bits per sample. The step size is

$$\Delta = \frac{2A_{\max}}{L}.$$

Worked Example: $B = 3$ bits

$B = 3 \Rightarrow L = 2^3 = 8$ levels, and $\Delta = \dfrac{2A_{\max}}{8} = \dfrac{A_{\max}}{4}$. Each sampled value is rounded to the nearest quantization level $x_q[n]$, and each level is represented by a 3-bit binary word: codes range from `000` to `111`, i.e. 8 possible codes, one per level.

Key points on quantization.

- Quantization converts a continuous amplitude range into a finite number of discrete levels.
- The resolution of quantization depends on the number of bits B.
- Higher $B \Rightarrow$ more levels $\Rightarrow$ lower quantization error $\Rightarrow$ better accuracy.
- Encoding converts each quantized level into a binary code for digital processing, storage, and transmission.

To Derive / Work Out

- *Derive $\Delta = 2A_{\max}/L$ from first principles: the full range $2A_{\max}$ is split into L equal-width bins.*
- *Work out the quantization error bound: for round-to-nearest quantization, the maximum error per sample is $\Delta/2$. Express this in terms of B and $A_{\max}$, and note how it shrinks as B increases (motivating the 6 dB/bit SNR improvement, to be formalized later in the course).*
- *Reproduce the $B = 3$ example table yourself: list all 8 levels ($-7\Delta/2, \ldots, +7\Delta/2$ in the convention used) alongside their 3-bit binary codes.*

1.8.4 Digital-to-Analog Conversion (DAC)

Pipeline (reverse direction): digital input $b[n] \rightarrow$ digital signal (encoded) $\rightarrow$ DAC (digital-to-analog converter) $\rightarrow$ staircase output $x_{DAC}(t)$ (zero-order hold, ZOH) $\rightarrow$ reconstruction via an analog low-pass filter $H(f) \rightarrow$ analog output $x_a(t)$ (continuous signal).

Key Result: Ideal reconstruction condition

The ideal low-pass reconstruction filter removes spectral images and smooths the staircase signal, provided

$$f_c \leq \frac{f_s}{2}.$$

The full DAC pipeline, as in the notes, has six panels: (1) a digital input sequence (e.g. 4-bit, levels 0–15, 16 possible codes); (2) the staircase (zero-order-hold) DAC output $x_{DAC}(t)$, held for T_s seconds per sample; (3) an ideal reconstruction low-pass filter with cutoff $f_c \leq f_s/2$; (4) the filtered analog output; (5) the overall time-domain relationship (digital input, staircase output, reconstructed output overlaid); (6) the frequency-domain view: baseband spectrum plus images (spectral replicas) centered at kf_s, $k = 1, 2, \ldots$, where the reconstruction filter must pass the baseband and reject the images.

⇄ Connections

The phrase "spectral images centered at kf_s" is exactly the frequency-domain shadow of the aliasing phenomenon this book derives in the time/digital-frequency domain in § 3.2.2. The ADC pipeline (§ 1.8) and this DAC pipeline are mirror images of one another:

$$x_a(t) \xrightarrow{\text{ADC}} b[n] \xrightarrow{\text{DSP algorithm}} b'[n] \xrightarrow{\text{DAC}} x'_a(t).$$

To Derive / Work Out

- *Explain why sampling in time creates periodic images of the spectrum at kf_s (connects to the Poisson summation formula / sampling theorem, formalized once DFT/DTFT and Z-transform are covered later in the course).*
- *Justify the reconstruction condition $f_c \leq f_s/2$: why must the low-pass cutoff sit at or below the Nyquist frequency to isolate the baseband image without contamination from the first replica?*
- *Reconcile the ADC pipeline and the DAC pipeline as mirror images of each other, and identify which stage in each half corresponds to which in the other.*

Bridging to Chapter 2

This chapter established *why* an analog signal is turned into a sequence of numbers $x(n) = x_a(nT_s)$, and the physical stages (sample, hold, quantize, encode) that do it. From here on, the book works almost exclusively with the resulting object: a **discrete-time sequence** $x(n)$. Chapter 2 takes up the question of how to notate, manipulate, and classify such sequences — the first step toward treating them as inputs to systems.

Chapter 2

Signal Representation, Operations, and Classification

Where we are. Chapter 1 established that a physical signal is sampled, held, quantized, and encoded into a discrete-time sequence $x(n)$. **What this chapter does.** It develops the vocabulary and algebra needed to work with such sequences: how to write them down, how to combine and transform them (addition, multiplication, folding, shifting), the elementary building-block sequences ($\delta(n)$, $u(n)$), even/odd decomposition, periodicity, boundedness/summability, and the exponential and sinusoidal families. **Where it leads.** The impulse-train representation built in § 2.8 is the single most important idea in this chapter — it is the starting point for the entire convolution-sum derivation in Ch. 4. The "why (not) DSP" motivation opens the chapter and answers the open question posed at the end of Ch. 1.

2.1 Why and Why Not DSP?

⇄ Connections

This section directly answers the open question posed at the end of § 1.1 of Ch. 1: *why not only analog, or only digital?*

At a glance, the signal taxonomy from § 1.3 refines further:

$$\text{Signal} \to \{\text{Analog}, \text{Digital}\}, \qquad \text{Analog} \to \{\text{CT (continuous-time)}, \text{DT (discrete-time)}\}.$$

(Analog splits further into CT and DT; "digital" means discrete in both time and amplitude, consistent with table 1.1.)

2.1.1 Why DSP?

- Minimal sensitivity to component tolerance and environmental changes (noise).
- Volume production without adjustment.
- Digital circuits are fully integrable.
- With word-length (bits), DSP accuracy increases — and so does cost (more bits ⇒ more accurate value).
- Dynamic range of DSP is almost unlimited.
- A DSP system can perform multiple tasks together, unlike ASP (analog signal processing).
- DSP processor characteristics are adjustable (e.g. adaptive filters).

- DSP provides exact linear phase (no delay distortion) systems.
- DSP provides multirate signal processing (§ 3.4).
- DSP has no loading problem due to cascading (unlike analog stages).
- Digital signals can be stored on magnetic tape (or other media) for a long time without degradation.
- DSP can process very low-frequency signals without issues.

Definition: Dynamic range

$$DR = \frac{\text{Strongest value}}{\text{Weakest value}}.$$

Time Division Multiplexing (TDM) — pictorial summary. Multiple input sources (e.g. voice, video, data) are each assigned a fixed time slot ($T_1, T_2, T_3, \dots$) within a repeating frame. A MUX interleaves samples from each source into one high-speed stream over a single communication channel; a DEMUX at the receiver uses timing information to separate the stream back into the original signals. Many signals share one channel by using different time slots; only one source transmits at a time, though all appear to communicate simultaneously; this requires precise synchronization between sender and receiver, and is used in digital telephony, computer networks, and satellite links.

2.1.2 Why Not DSP?

- Increased complexity (more/different components: ADC, DAC, processor, anti-aliasing/reconstruction filters — recall § 1.8).
- Frequency range of operation is limited (sampling rate is limited).
- Power dissipation.

To Derive / Work Out

For each "why not" point, connect it back to a concept from Ch. 1: e.g. the limited frequency range is exactly the Nyquist constraint on f_s from the sampling discussion — write out the precise relationship once § 3.2 is reached.

2.2 Signal Representation

A discrete-time signal can be represented in several equivalent ways: **sequentially** (a list of sample values in order), **tabularly** (a table of n vs. $x(n)$), or **graphically** (a stem plot of $x(n)$ against n). Signals may be real- or complex-valued.

Sequential notation, with the origin marked by an arrow:

$$x(n) = \{1, 2, 3, 4, 5, 6\}, \qquad x(n) = \Big\{-1,\ 2,\ \underset{\substack{\uparrow\\ n=0}}{3},\ 1,\ 4,\ 5\Big\}.$$

The arrow under a sample marks which index corresponds to $n = 0$; indices to its left/right are read off as $n = -1, -2, \dots$ and $n = 1, 2, \dots$ respectively.

Worked Example: piecewise signal

$$x(n) = \begin{cases} n, & n > 0 \\ -n, & \text{otherwise} \end{cases} \qquad \text{(i.e. } x(n) = |n|\text{)}.$$

2.2.1 Sampling Notation Recap

$$x_a(t) \xrightarrow{\text{Sampler}} x_a(nT), \ -\infty < n < \infty, \quad T_s = \frac{1}{f_s} \text{ (sampling period)},$$

$$x_a(t) \xrightarrow{\text{A/D}} x(n) \text{ or } x[n]$$

(the two notations $x(n)$, $x[n]$ are used interchangeably).

⇄ Connections

This is exactly the notation fixed in § 1.8; from here on the book uses $x(n)$ throughout, reserving $x_a(t)$ for the underlying continuous-time signal when a distinction is needed (as in § 3.1.3).

2.2.2 Complex-Valued Signals

A real sequence can be relabeled with complex values, e.g.

$$x(n) = \{1, 2, 3, 4, 5\} \longrightarrow \{-1 + j,\ j,\ j + 2,\ +5j\}$$

(each real sample reinterpreted as a complex number; here $j + 2$ denotes $2 + j$).

To Derive / Work Out

Practice going back and forth between sequential and graphical (stem-plot) forms for a few sequences of your own, always marking the $n = 0$ arrow. For a complex sequence, separately plot $\text{Re}\{x(n)\}$ *and* $\text{Im}\{x(n)\}$ *against n.*

2.3 Operations on Signals

2.3.1 Addition and Multiplication

For two sequences $x(n)$ and $y(n)$ defined over a common index range (pad with zeros where one is undefined),

$$z(n) = x(n) \pm y(n) \quad \text{(addition/subtraction, sample-by-sample)},$$
$$z(n) = x(n) \cdot y(n) \quad \text{(multiplication, sample-by-sample — not convolution)}.$$

To Derive / Work Out

Work a concrete example: align $x(n)$ and $y(n)$ on a common index axis (padding with 0s outside each one's support), then compute $z(n) = x(n) + y(n)$ and $z(n) = x(n) \cdot y(n)$ term by term. Double-check index alignment carefully — the most common source of error in these problems.

2.3.2 Folding (Time Reversal) and Shifting

Definition: Folding / time reversal

$$x(n) \longrightarrow x(-n).$$

Worked Example: folding

If $x(n) = \{1, 2, \underline{0}, 3, 4\}$ (origin at the underlined 0), then $x(-n) = \{4, 3, \underline{0}, 2, 1\}$ — the sequence is reversed about $n = 0$.

Definition: Delay and advance

$$x(n-1) = \text{delay by } 1, \qquad x(n+1) = \text{advance by } 1.$$

General shift by k: $x(n) \xrightarrow{D/A} x(n-k)$, with $k > 0$ a delay and $k < 0$ an advance.

Combining fold and shift (applying a shift after a fold) gives $x(n) \to x(-n-k)$: first reflect about the origin, then the sign and placement of k determine whether the result is a further delay or advance of the folded sequence. Folding and shifting do *not* commute in the naive sense — always track what happens to the origin marker.

To Derive / Work Out

Take $x(n) = \{-1, 2, \underline{1}, 3, -2\}$ with the origin marked at the middle sample. Compute and plot, step by step: $x(n-1)$ and $x(n+1)$; $x(n-2)$ and $x(n-3)$ (note how zero-padding appears at the start as the causal window shifts past the original support); $x(-n)$ (fold) and compare its support/shape to the original. Confirm in each case which sample sits at $n = 0$ after the operation.

⇄ Connections

Folding and shifting are not idle algebra: Chs. 4 and 5 show that computing a convolution sum by hand is *literally* "fold, shift, multiply, add," and that correlation is the same recipe with the fold step removed. Everything practiced here is directly reused there.

2.4 Elementary Signals

Definition: Unit impulse and unit step

$$\delta(n) = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases} \qquad u(n) = \begin{cases} 1, & n \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Key Result: Interrelations

$$u(n) = \sum_{k=0}^{\infty} \delta(n-k), \qquad \delta(n) = u(n) - u(n-1), \qquad \delta(n) = u(n) \cdot u(-n).$$

(The last identity holds because $u(n) = 1$ for $n \geq 0$ and $u(-n) = 1$ for $n \leq 0$; their product is 1 only at $n = 0$.)

To Derive / Work Out

Facts to verify: (1) $u(n) + u(-n) = 1 + \delta(n)$ for all n. (2) $u(n) + u(-n-1) = 1$ for all n. (3) $u(n) - u(-n) = ?$ (4) $u(n) - u(-n-1) = ?$ Verify (1) and (2) by checking three cases $n > 0$, $n = 0$, $n < 0$, substituting the piecewise definitions directly. Then work out (3) and (4) the same way — these are "signum-like" identities and should come out to sgn*-type expressions.*

2.5 Even and Odd Signals

Definition

Even (symmetric): $x_e(n) = x_e(-n)$. Odd (antisymmetric): $x_o(n) = -x_o(-n)$.

Key Result: Even–odd decomposition

Any signal $x_m(n)$ can be written as $x_m(n) = x_e(n) + x_o(n)$, where

$$x_e(n) = \frac{x_m(n) + x_m(-n)}{2}, \qquad x_o(n) = \frac{x_m(n) - x_m(-n)}{2}.$$

To Derive / Work Out

Derive the decomposition formulas yourself: write $x_m(-n) = x_e(-n) + x_o(-n) = x_e(n) - x_o(n)$ (using the symmetry definitions), then solve the two simultaneous equations $x_m(n) = x_e(n)+x_o(n)$ and $x_m(-n) = x_e(n) - x_o(n)$ for $x_e(n)$ and $x_o(n)$. Apply the formulas to a sequence of your own (mark the origin first!) and confirm $x_e(n)$ is symmetric and $x_o(n)$ is antisymmetric about $n = 0$.

2.6 Periodicity, Boundedness, and Summability

Definition: Periodic signal

$$x(n) = x(n + N), \qquad N = 1, 2, 3, \ldots$$

(N is a period of the signal; the smallest such N is the fundamental period.)

Definition: Bounded signal

$$|x(n)| \le M < \infty \text{ for all } n$$

(for some finite M, not depending on n).

Definition: Summability and energy/power classes

1. Absolutely summable: $\displaystyle\sum_{n=-\infty}^{\infty} |x(n)| \le P < \infty$.

2. Square summable (finite energy $\Rightarrow$ energy signal): $\displaystyle\sum_{n=-\infty}^{\infty} |x(n)|^2 \le Q < \infty$.

3. Average power: for a periodic signal with period N,

$$P_{\text{avg}} \triangleq \frac{1}{N} \sum_{n=0}^{N-1} |x(n)|^2;$$

more generally, for an arbitrary (possibly non-periodic) signal,

$$P_{\text{avg}} \triangleq \lim_{K\to\infty} \frac{1}{2K+1} \sum_{n=-K}^{K} |x(n)|^2.$$

To Derive / Work Out

Explain why every absolutely summable signal is also square summable ($\ell^1 \subset \ell^2$ for bounded sequences, via Cauchy–Schwarz), but not conversely. Classify a few standard sequences (e.g. $x(n) = 1/n$ for $n \ge 1$, $x(n) = (1/2)^n u(n)$, $x(n) = \cos(\omega_0 n)$) as bounded/unbounded, absolutely summable or not, square summable or not, and compute P_{avg} where finite. This builds the "energy signal vs. power signal" vocabulary used throughout the course.

2.7 Exponential and Sinusoidal Signals

Definition: Two-sided vs. one-sided exponential

Two-sided: $x(n) = A\alpha^n$. One-sided: $x(n) = A\alpha^n u(n)$. For the one-sided (causal) exponential: $|\alpha| > 1 \Rightarrow$ grows without bound (diverges); $|\alpha| \leq 1 \Rightarrow$ bounded (does not diverge).

2.7.1 Periodicity of a Discrete-Time Sinusoid

Requiring $x(n) = x(n+N)$ for $x(n) = \cos(\omega_0 n + \phi)$:

$$\cos(\omega_0 n + \phi) = \cos\big(\omega_0(n+N) + \phi\big) \implies \omega_0 N = 2\pi\lambda \ (\lambda \in \mathbb{Z}) \implies \frac{\omega_0}{2\pi} = \frac{\lambda}{N}.$$

Key contrast with continuous time

A discrete-time sinusoid is periodic *iff* $\omega_0/2\pi$ is rational (some ratio λ/N of integers) — unlike continuous time, where every sinusoid is periodic.

Worked Example: $x(n) = \cos(3n)$

Matching to $\cos(\omega_0 n)$ gives $\omega_0 = 3$, so $\omega_0/2\pi = 3/(2\pi)$. Since $3/(2\pi)$ is irrational, $x(n) = \cos(3n)$ is *not* periodic.

To Derive / Work Out

Contrast this with $x(n) = \cos(3\pi n/7)$: here $\omega_0 = 3\pi/7$, so $\omega_0/2\pi = 3/14$, which is rational $\Rightarrow$ the signal is periodic with fundamental period $N = 14$ (after canceling any common factors between λ and N). Work through this example fully, finding the fundamental period.

⇄ Connections

Section 3.1.3 in the next chapter defines the digital (normalized) frequency $\omega_0 = \Omega T = 2\pi f/f_s$ that arises from sampling a continuous-time sinusoid, and shows that its principal range $-\pi < \omega_0 \leq \pi$ is precisely the discrete-time face of the Nyquist limit. The periodicity condition derived here ($\omega_0/2\pi \in \mathbb{Q}$) is reused verbatim once ω_0 is tied to a physical sampling rate.

2.7.2 Periodicity of a Sum of Periodic Signals

Given $x_1(n), x_2(n), x_3(n)$ periodic with periods N_1, N_2, N_3, and $x_4(n) = x_1(n) + x_2(n) + x_3(n)$:

$$N_4 = \mathrm{lcm}(N_1, N_2, N_3).$$

Worked Example: lcm example

$N_1 = 3, N_2 = 5, N_3 = 12 \ \Rightarrow \ N_4 = \mathrm{lcm}(3, 5, 12) = 60.$

To Derive / Work Out

Verify $\mathrm{lcm}(3, 5, 12) = 60$ *via prime factorization* ($3 = 3$, $5 = 5$, $12 = 2^2 \cdot 3$; *the lcm takes the highest power of each prime:* $2^2 \cdot 3 \cdot 5 = 60$). *Think about why the LCM is the right operation: each component signal repeats exactly at multiples of its own period, so the sum repeats only once all components simultaneously complete a whole number of cycles.*

2.7.3 The Delay System

Definition: Unit delay and cascaded delays

$$\text{Delay} = z^{-1}: \quad x(n) \longrightarrow \boxed{z^{-1}} \longrightarrow y(n) = x(n-1).$$

N-fold delay: $x(n) \to \boxed{z^{-N}} \to y(n) = x(n-N)$, equivalently realized as a cascade of N unit delays:

$$x(n) \to \boxed{z^{-1}} \to \boxed{z^{-1}} \to \cdots \to \boxed{z^{-1}} \to y(n).$$

To Derive / Work Out

This is your first encounter with the z^{-1} operator as "delay by one sample." Keep this notation in mind — it becomes the building block for the Z-transform and for realizing FIR/IIR filters later in the course.

2.8 Expressing Any Signal Using Elementary Signals

Key Result: General impulse representation

Any discrete-time signal can be written as a weighted sum of shifted unit impulses:

$$x(n) = \sum_{k=-\infty}^{\infty} x(k)\,\delta(n-k).$$

Worked Example: impulse-train form

$$x(n) = \{2, 4, 1\} = 2\delta(n) + 4\delta(n-1) + \delta(n-2),$$
$$x(n) = \{2, 5, 4, 7\} = 2\delta(n) + 5\delta(n-1) + 4\delta(n-2) + 7\delta(n-3),$$
$$x(n) = \{0, 1, 2, 3\} = \delta(n-1) + 2\delta(n-2) + 3\delta(n-3).$$

(Origin marked at the first listed sample, $n = 0$, in each case.)

⇄ Connections

This impulse-train identity is the single most-reused fact in the whole book. Chapter 3 calls it "the baseband (impulse-train) representation" and uses it, together with linearity and time-invariance, to justify why an LTI system is completely characterized by its impulse response $h(n)$. Chapter 4 makes this precise: Steps 4–5 of the convolution-sum derivation (§ 4.2) literally substitute this identity into the superposition principle to produce the convolution sum $y(n) = \sum_k x(k)h(n-k)$. Chapter 5 then reuses the same identity to motivate the correlation sum by analogy. Every occurrence after this point is the *same* formula being put to a new use — it is worth memorizing exactly as stated above.

Rewriting the impulse sum in terms of unit steps. Since $\delta(n-k) = u(n-k) - u(n-k-1)$, substituting into $x(n) = \sum_k x(k)\delta(n-k)$ and collecting terms by common $u(n-k)$ gives a telescoping representation: the coefficient of $u(n-k)$ is $x(k)-x(k-1)$ (with $x(k) = 0$ outside the signal's support).

Worked Example: telescoping step-function form

$$x(n) = \{2, 4, 1\} \;\Rightarrow\; x(n) = 2u(n) + 2u(n-1) - 3u(n-2) - u(n-3)$$
(coefficients: $2,\ 4-2=2,\ 1-4=-3,\ 0-1=-1$),

$$x(n) = \{2, 5, 4, 7\} \;\Rightarrow\; x(n) = 2u(n) + 3u(n-1) - u(n-2) + 3u(n-3) - 7u(n-4)$$
(coefficients: $2,\ 5-2=3,\ 4-5=-1,\ 7-4=3,\ 0-7=-7$).

To Derive / Work Out

Derive the telescoping rule formally: substitute $\delta(n-k) = u(n-k) - u(n-k-1)$ into $\sum_k x(k)\delta(n-k)$, re-index the second sum ($k \to k+1$), and combine like terms in $u(n-k)$ to confirm the coefficient is $x(k) - x(k-1)$. Apply the rule to $x(n) = \{0, 1, 2, 3\}$ (origin at the first sample) and write out the step-function form yourself; check it against direct substitution of the impulse form. Keep this technique in your toolkit — it is the standard way to express a finite-length sequence as a combination of shifted steps rather than shifted impulses, useful for later system-response calculations (step response vs. impulse response, § 3.8).

Bridging to Chapter 3

This chapter equipped discrete-time sequences with an algebra: addition, multiplication, folding, shifting, and — most importantly — the impulse-train representation (§ 2.8). Chapter 3 now asks what happens when such a sequence is fed *into* a system: it formalizes the sampling theorem hinted at in § 1.8, and then treats a discrete-time system as an operator with properties (linearity, causality, stability) that will let the impulse-train identity do real work.

Chapter 3

Discrete-Time Signals and Systems

Where we are. Chapter 2 gave us the algebra of sequences, culminating in the impulse-train representation $x(n) = \sum_k x(k)\delta(n-k)$ (§ 2.8). **What this chapter does.** Two things, in order. First (§ 3.2), it makes precise the informal sampling inequality flagged in § 1.8, states the Nyquist–Shannon theorem, and works a full aliasing example. Second (§ 3.3 onward), it treats a discrete-time system as an operator $T[\cdot]$ and develops the five properties every system in this course will be checked against: linearity, causality, static/dynamic character, stability, and the impulse/step response. **Where it leads.** The impulse response $h(n)$ defined in § 3.8 is, together with linearity and time-invariance, exactly what Ch. 4 needs to derive the convolution sum.

3.1 From Continuous to Discrete: The Baseband Signal

3.1.1 Why We Sample

Physical signals (speech, sensor voltages, radio waves) are naturally continuous-time functions $x(t)$. A digital system cannot store or process a continuum of values — it can only store numbers at discrete time instants. Sampling takes the value of $x(t)$ at a discrete, uniformly spaced set of times $t = nT$, where T is the sampling period and $n \in \mathbb{Z}$ is an integer index.

3.1.2 Representing a Discrete Signal with Impulses

⇄ Connections

This is precisely § 2.8's impulse-train representation, restated here as it is about to be put to use:

$$x(n) = \sum_{k=-\infty}^{\infty} x(k)\,\delta(n-k), \qquad \delta(n) = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}.$$

This is called the **baseband (impulse-train) representation** of a discrete signal — the discrete analogue of the sifting property of the continuous-time impulse. It lets us build any input as a sum of impulses, and hence (by linearity) express any output as a sum of scaled, shifted impulse responses. This is the seed of the convolution sum derived in full in § 4.2.

3.1.3 Sampling a Sinusoid: Continuous → Discrete Frequency

Let a continuous-time sinusoid be $x(t) = A\cos(\Omega t + \phi)$, where Ω is the angular frequency (rad/s) and $f = \Omega/2\pi$ is the cyclic frequency (Hz). Sampling at $t = nT$ (so $T = 1/f_s$, f_s the sampling rate) gives

$$x(n) = x(t)\big|_{t=nT} = A\cos(\Omega nT + \phi).$$

Key Result: Digital (normalized) frequency

$$\omega_0 \triangleq \Omega T = \frac{\Omega}{f_s} = 2\pi \frac{f}{f_s}, \qquad \text{so that} \qquad x(n) = A\cos(\omega_0 n + \phi).$$

- Ω: analog angular frequency, units rad/s.
- $f = \Omega/2\pi$: cyclic frequency, units Hz.
- $\omega_0 = \Omega T = 2\pi f/f_s$: digital (normalized) frequency — dimensionless, in "radians per sample." It tells you how many radians the phase advances from one sample to the next, not per second.

Why ω_0 is restricted to $(-\pi, \pi]$. Because $\cos(\omega_0 n + \phi)$ is evaluated only at integer n, adding any multiple of 2π to ω_0 produces exactly the same sequence:

$$\cos\big((\omega_0 + 2\pi)n + \phi\big) = \cos(\omega_0 n + 2\pi n + \phi) = \cos(\omega_0 n + \phi).$$

So all distinct discrete-time sinusoids are obtained from ω_0 ranging over any 2π-length interval; by convention, $0 \le \omega_0 < \pi$ or equivalently $-\pi < \omega_0 \le \pi$. Here $\omega_0 = \pi$ corresponds to the fastest possible oscillation a discrete sequence can show ($x(n) = A\cos(\pi n + \phi) = A(-1)^n \cos\phi$, alternating sign every sample). This upper bound of π is precisely the discrete-time face of the Nyquist limit, made precise next.

⇄ Connections

Section 2.7.1 showed a discrete-time sinusoid $\cos(\omega_0 n + \phi)$ is periodic iff $\omega_0/2\pi$ is rational. That derivation used a purely abstract ω_0; here ω_0 is tied to a physical sampling rate via $\omega_0 = 2\pi f/f_s$, so the periodicity question becomes a question about whether f/f_s is rational.

3.2 The Sampling Theorem and Aliasing

3.2.1 Statement of the Theorem

Key Result: Nyquist–Shannon Sampling Theorem

A continuous-time signal $x(t)$ band-limited to a maximum frequency $f_{\max}$ (no frequency component above $f_{\max}$ Hz) can be perfectly reconstructed from its samples $x(nT)$ if and only if the sampling frequency satisfies

$$f_s > 2f_{\max}.$$

The quantity $2f_{\max}$ is the **Nyquist rate**, and $f_s/2$ is the **Nyquist frequency**.

In terms of the digital frequency of § 3.1.3, the condition $f \le f_s/2$ is exactly what keeps $\omega_0 = 2\pi f/f_s \le \pi$, i.e. keeps ω_0 inside its principal range. This is why $-\pi < \omega_0 < \pi$ (or $0 < \omega_0 < \pi$) is called the **principal / unambiguous range** of digital frequency: every physically distinguishable discrete-time frequency lives here.

⇄ Connections

This formalizes the inequality $f_s \gtrless 2f_0$ flagged provisionally in § 1.8 of Ch. 1. It also explains, in the frequency domain, the spectral images at kf_s mentioned in the DAC pipeline discussion (§ 1.8): those images are exactly what the reconstruction filter's cutoff $f_c \le f_s/2$ is designed to reject.

3.2.2 Aliasing: What Happens If You Violate Nyquist

If $f > f_s/2$, the sampled sequence becomes indistinguishable from the sample sequence of a lower-frequency sinusoid. This is **aliasing**: a high frequency "disguises itself" (becomes an alias of) a lower

one once sampled too slowly.

Key Result: Aliasing condition

Two continuous sinusoids of frequency f_1 and f_2 produce identical samples at rate f_s whenever

$$f_2 = f_1 + kf_s, \qquad k \in \mathbb{Z},$$

because this makes their digital frequencies differ by a multiple of 2π.

Worked Example: three-tone aliasing

Consider three continuous sinusoids with cyclic frequencies $f = 3, 7, 13$ Hz:

$$x_1(t) = \cos(6\pi t), \quad x_2(t) = \cos(14\pi t), \quad x_3(t) = \cos(26\pi t)$$

(using $\Omega = 2\pi f$).
Case A: sample at $f_s = 10$ Hz ($T_s = 0.1$ s). Using $\omega_0 = \Omega T_s = 2\pi f/f_s$:

$$\begin{aligned} x_1(n) &= \cos(0.6\pi n), \\ x_2(n) &= \cos(1.4\pi n) = \cos\big((2\pi - 0.6\pi)n\big) = \cos(0.6\pi n), \\ x_3(n) &= \cos(2.6\pi n) = \cos\big((2\pi + 0.6\pi)n\big) = \cos(0.6\pi n). \end{aligned}$$

All three completely different analog tones (3, 7, 13 Hz) produce the *exact same* discrete sequence $\cos(0.6\pi n)$ once sampled at 10 Hz. Once you have only the samples, you cannot tell which of the three produced them — this is aliasing. Note $7 = 10 - 3$ and $13 = 10 + 3$, i.e. $f_s \pm f_1$: exactly the aliasing condition $f_2 = f_1 + kf_s$.
Case B: sample faster, at $f_s = 50$ Hz ($T_s = 0.02$ s).

$$x_1(n) = \cos(0.12\pi n), \quad x_2(n) = \cos(0.28\pi n), \quad x_3(n) = \cos(0.52\pi n).$$

Now all three digital frequencies are distinct and lie in $(0, \pi)$ — the three tones are no longer confused. (Check: Nyquist frequency is $f_s/2 = 25$ Hz, and the largest input frequency, 13 Hz, is safely below it: $2f_{\max} = 26 < 50 = f_s$ satisfies the sampling theorem.)

Remark

Anti-aliasing rule of thumb: before sampling, pass the analog signal through a low-pass filter that removes all content above $f_s/2$ (an *anti-aliasing filter*), so that no out-of-band energy can fold back and corrupt the baseband spectrum. This is precisely the "Anti-aliasing Filter" block in the end-to-end DSP system diagram of § 1.8.

3.3 Discrete-Time Systems: Basic Framework

3.3.1 System as an Operator

Definition: Discrete-time system

A discrete-time system (DS) is any rule/operator $T[\cdot]$ mapping an input sequence $x(n)$ to an output sequence $y(n)$:

$$y(n) = T[x(n)], \qquad \text{written compactly as } y = f(x).$$

When we build a model or approximation $\hat{T}$ of a real system T, we quantify the mismatch with the error signal $e(n) = y(n) - \hat{y}(n)$, and $e(n) \equiv 0 \iff y(n) = \hat{y}(n)$: a model is exact exactly when

its error sequence is identically zero for every n. This is the basic idea behind system identification and model validation.

3.3.2 Worked Example: The Accumulator

Worked Example: Accumulator

Let $x(n) = \{1, 2, 3, 4, 5\}$ (origin at the first sample) and define

$$y(n) = \sum_{\ell=-\infty}^{n} x(\ell)$$

(the *accumulator*, or running sum: it adds up every input sample received so far).
Deriving a recursive (difference-equation) form. Split the sum at $\ell = -1$:

$$y(n) = \underbrace{\sum_{\ell=-\infty}^{-1} x(\ell)}_{=y(-1)} + \sum_{\ell=0}^{n} x(\ell).$$

Separate the last term of the second sum ($\ell = n$) from the rest: $\sum_{\ell=0}^{n} x(\ell) = \sum_{\ell=0}^{n-1} x(\ell) + x(n)$. But $y(-1) + \sum_{\ell=0}^{n-1} x(\ell) = y(n-1)$ by the definition of y. Hence

$$\boxed{y(n) = y(n-1) + x(n)}$$

— the accumulator's recursive (IIR-type) description: "new running total = old running total + new sample." It requires one initial condition $y(-1)$ (for a causal, initially-at-rest system, $y(-1) = 0$).
Computing sample by sample (assume $y(-1) = 0$):

$$y(0) = 1,\ y(1) = 1 + 2 = 3,\ y(2) = 3 + 3 = 6,\ y(3) = 6 + 4 = 10,\ y(4) = 10 + 5 = 15.$$

So $y(n) = \{1, 3, 6, 10, 15\}$ — the cumulative sums of $x(n)$.

⇄ Connections

The accumulator is the running example of this chapter: it returns to illustrate linearity (§ 3.5), stability (§ 3.7), and the impulse/step response (§ 3.8), where its impulse response is shown to be $h(n) = u(n)$ and its step response $s(n) = (n+1)u(n)$.

3.4 Characterization / Classification of Systems by Operation

3.4.1 Rate of Sampling: Up-sampling and Down-sampling

Definition: Up-sampling by L

$$y(n) = \begin{cases} x\left(\dfrac{n}{L}\right), & n = 0, \pm L, \pm 2L, \dots \\ 0, & \text{otherwise} \end{cases}$$

L is the up-sampling factor: $L - 1$ zeros are inserted between consecutive input samples, so the output runs L times "faster."

Worked Example: $L = 3$

Let $x(n) = \{-1, \underline{1}, 2\}$ (origin at the underlined sample), i.e. $x(0) = 1$, $x(-1) = -1$, $x(1) = 2$. Since only $n = 0, \pm 3, \pm 6, \dots$ can be nonzero:

$$y(0) = x(0) = 1, \quad y(3) = x(1) = 2, \quad y(-3) = x(-1) = -1, \quad y(\pm 1) = y(\pm 2) = 0.$$

So $y(n) = \{\dots, 0, 0, \underset{n=-3}{-1}, 0, 0, \underset{n=0}{1}, 0, 0, \underset{n=3}{2}, 0, 0, \dots\}$.

Definition: Down-sampling (decimation) by M

$$y(n) = x(nM), \qquad M = 1, 2, 3, \dots$$

Only every M-th sample of x is kept; the rest are discarded. The output runs M times "slower."

Worked Example: $M = 2$

Let $x(n) = \{-1, -2, \underline{1}, 3, 4, 1, 5\}$ (origin at the underlined sample), i.e. $x(-2) = -1, x(-1) = -2, x(0) = 1, x(1) = 3, x(2) = 4, x(3) = 1, x(4) = 5$. Then $y(n) = x(2n)$:

$$y(-1) = x(-2) = -1, \quad y(0) = x(0) = 1, \quad y(1) = x(2) = 4, \quad y(2) = x(4) = 5.$$

So $y(n) = \{-1, \underline{1}, 4, 5\}$.

Remark

Up-sampling followed by an appropriate low-pass ("anti-imaging") filter is the standard way to interpolate a discrete signal to a higher rate; down-sampling preceded by an anti-aliasing low-pass filter (recall § 3.2.2!) is the standard way to decimate to a lower rate without introducing aliasing. Together, $\uparrow L$ + filter + $\downarrow M$ implement rational sample-rate conversion by L/M.

3.4.2 Moving Average (MA) Filter

Definition: M-point moving average

$$y(n) = \frac{1}{M} \sum_{k=0}^{M-1} x(n-k).$$

Each output sample is the average of the current input sample and the previous $M - 1$ input samples — a simple low-pass/smoothing filter, widely used for noise reduction.

⇄ Connections

The moving-average filter reappears in § 3.7 as the canonical example of a *finite*-memory system that is BIBO stable, in direct contrast with the (infinite-memory) accumulator of § 3.3.2, which is not.

3.5 Linearity

3.5.1 Definition

Definition: Linearity

A system T with $x_1(n) \to y_1(n) = T[x_1(n)]$ and $x_2(n) \to y_2(n) = T[x_2(n)]$ is **linear** iff it satisfies both:

1. **Homogeneity (scaling):** $T[\alpha x(n)] = \alpha\, T[x(n)]$ for any constant α.
2. **Additivity (superposition):** $T[x_1(n) + x_2(n)] = T[x_1(n)] + T[x_2(n)]$.

Equivalently, the single combined superposition test:

$$T\big[a_1x_1(n) + a_2x_2(n)\big] = a_1T[x_1(n)] + a_2T[x_2(n)] = a_1y_1(n) + a_2y_2(n)$$

for all constants a_1, a_2 and all input pairs. (Setting $a_2 = 0$ recovers homogeneity as a special case, which is why the single combined test is both necessary and sufficient.)

⇄ Connections

This superposition test is exactly what Ch. 4 needs. The convolution-sum derivation (§ 4.2) applies homogeneity to a single term $x(k)\delta(n-k)$ and then additivity summed over *all* k — i.e. it is the infinite-sum extension of precisely this definition.

3.5.2 Worked Examples

Worked Example: the Accumulator is linear

$y(n) = \sum_{\ell=-\infty}^{n} x(\ell)$. Test with $x(n) = \alpha x_1(n) + \beta x_2(n)$:

$$T[\alpha x_1 + \beta x_2] = \sum_{\ell=-\infty}^{n} \big(\alpha x_1(\ell) + \beta x_2(\ell)\big) = \alpha \sum_{\ell=-\infty}^{n} x_1(\ell) + \beta \sum_{\ell=-\infty}^{n} x_2(\ell) = \alpha y_1(n) + \beta y_2(n).$$

Since summation distributes over addition and scalar multiplication, the accumulator satisfies superposition exactly ⇒ linear. (More generally, the recursive form $y(n) = y(n-1) + x(n)$, with zero initial conditions, is a linear difference equation.)

Worked Example: $y(n) = n\,x(n)$ is linear

Let $y_1(n) = n\,x_1(n)$, $y_2(n) = n\,x_2(n)$. Then

$$T[a_1x_1(n) + a_2x_2(n)] = n\big(a_1x_1(n) + a_2x_2(n)\big) = a_1n\,x_1(n) + a_2n\,x_2(n) = a_1y_1(n) + a_2y_2(n).$$

Superposition holds ⇒ linear, even though the system is *time-varying* (its multiplier depends explicitly on n). **Exam point:** linearity and time-invariance are independent properties — a system can be linear but not time-invariant (this one), or nonlinear but time-invariant.

Worked Example: $y(n) = x^2(n)$ is nonlinear

Test homogeneity alone: $T[\alpha x(n)] = (\alpha x(n))^2 = \alpha^2 x^2(n) = \alpha^2 T[x(n)] \neq \alpha T[x(n)]$ unless $\alpha \in \{0, 1\}$. Homogeneity already fails for general α, so the system is not linear.

Exam strategy for linearity questions

1. Write $y_1 = T[x_1]$, $y_2 = T[x_2]$ using the system's defining formula.
2. Form $x(n) = a_1x_1(n) + a_2x_2(n)$ and compute $T[x(n)]$ directly from the formula.
3. Compare with $a_1y_1(n) + a_2y_2(n)$ term by term.
4. If they match for all $a_1, a_2, x_1, x_2 \Rightarrow$ linear. If you can find even one counterexample (often just try $\alpha = 2$ on a simple sequence) $\Rightarrow$ nonlinear. Watch for squares/products of x, constants added independent of input (e.g. $y(n) = x(n) + 5$ is *not* linear despite looking simple — it fails homogeneity/additivity because of the added constant), absolute values, and nonlinear functions like $\log, \exp$.

3.6 Causality

3.6.1 Definition

Definition: Causal system

A system is **causal** if the output at any time n depends only on the input at the present and past times $(x(n), x(n-1), x(n-2), \dots)$ — never on future input values. Formally, $y(n) = f(x(n), x(n-1), x(n-2), \dots)$ is causal; a system needing $x(n+1), x(n+2), \dots$ as well is non-causal.

Equivalent formal test. T is causal iff: whenever two inputs agree up to time N, $x_1(n) = x_2(n)$ for all $n \le N$, then the outputs also agree up to time N: $y_1(n) = y_2(n)$ for all $n \le N$. In words: the past+present of the input completely determines the past+present of the output; future input cannot influence past/present output.

3.6.2 Worked Examples

System	Depends on	Verdict
$y(n) = x(n)$	present only	Causal
$y(n) = a\,x(n)$	present only (scaled)	Causal
$y(n) = x(2n)$	future samples (for $n > 0$, $2n > n$)	Non-causal
$y(n) = x(n^2)$	future samples (for $\lvert n\rvert > 1$)	Non-causal
$y(n) = x(n) + x(n+1)$	needs $x(n+1)$, a future sample	Non-causal

Check for $y(n) = x(2n)$: at $n = 3$, $y(3) = x(6)$ — the output at time 3 needs the input at (future) time 6. Non-causal.

3.6.3 Causal Signals

Definition: Causal signal

Do not confuse a causal *system* with a causal *signal*. A sequence $x(n)$ is **causal** if $x(n) = 0$ for all $n < 0$, i.e. it "starts" at (or after) $n = 0$ and has no values in negative time.

3.6.4 Static vs. Dynamic Systems

Definition: Static vs. dynamic

Static (memoryless): $y(n)$ depends only on $x(n)$ at the same instant n (no past or future samples, no internal state). Example: $y(n) = a\,x(n)$, $y(n) = x^2(n)$. **Dynamic (with memory):** $y(n)$ depends on $x(n)$ at other time instants too, or on stored past outputs. Example: the accumulator, the moving-average filter, $y(n) = x(n) + x(n-1)$.

Every static system is automatically causal; a dynamic system may or may not be causal depending on whether it looks into the future.

⇄ Connections

Causality returns with sharper teeth in § 4.5: there, "$h(n) = 0$ for $n < 0$" (a causal *system*) is shown to truncate the convolution sum's upper limit, and "$x(n) = 0$ for $n < 0$" (a causal *signal*, as just defined) truncates its lower limit — the two causality notions defined here combine into the finite-limit convolution sum used in almost every practical filtering problem.

3.7 Stability: Bounded-Input, Bounded-Output (BIBO)

3.7.1 Definition

Definition: BIBO stability

A system is **BIBO stable** if every bounded input produces a bounded output:

$$|x(n)| \le B_x < \infty \;\forall n \implies |y(n)| \le B_y < \infty \;\forall n,$$

for some finite constants B_x, B_y (which may depend on each other but not on n). If even one bounded input produces an unbounded output, the system is unstable.

3.7.2 Worked Examples

Worked Example: $y(n) = x^2(n)$

If $|x(n)| \le B_x$ for all n, then $|y(n)| = |x(n)|^2 \le B_x^2 < \infty$. Since B_x^2 is finite whenever B_x is finite, this system is BIBO stable (a memoryless, bounded nonlinearity of a bounded input stays bounded).

Worked Example: Moving Average

Suppose $|x(n)| \le B_x$ for all n. Using the triangle inequality,

$$|y(n)| = \left|\frac{1}{M}\sum_{k=0}^{M-1} x(n-k)\right| \le \frac{1}{M}\sum_{k=0}^{M-1} |x(n-k)| \le \frac{1}{M}\cdot M \cdot B_x = B_x.$$

Bounded input ⇒ bounded output (with $B_y = B_x$) ⇒ BIBO stable. This holds because the sum has a finite number of terms (M) with bounded, fixed coefficients $1/M$.

Worked Example: Accumulator is BIBO *unstable*

Take the simplest bounded input: the unit step $x(n) = u(n)$, $|x(n)| \le 1$ for all n (so $B_x = 1$).

But

$$y(n) = \sum_{\ell=0}^{n} u(\ell) = n + 1 \xrightarrow{n\to\infty} \infty.$$

The output grows without bound even though the input is bounded ⇒ the (infinite-memory) accumulator is BIBO unstable. Contrast with the moving average: a *finite*-length running sum is stable, but an *infinite*-length running sum (the full accumulator) is not — the number of terms being summed, and whether their coefficients are summable, is exactly what controls stability.

Key Result: General stability criterion for LTI systems (preview)

For a linear time-invariant (LTI) system with impulse response $h(n)$ (§ 3.8), BIBO stability is equivalent to absolute summability of the impulse response:

$$\sum_{n=-\infty}^{\infty} |h(n)| < \infty.$$

This single condition explains all three examples above: the moving-average filter has $h(n)$ nonzero over only M samples (finite sum ⇒ stable), while the accumulator has $h(n) = u(n)$, and $\sum_n |u(n)| = \sum_{n=0}^{\infty} 1 = \infty$ (not absolutely summable ⇒ unstable).

3.8 Impulse Response and Step Response

3.8.1 Definitions

Definition: Impulse response and step response

The **impulse response** $h(n)$ is the output of the system when the input is the unit impulse $\delta(n)$: $\delta(n) \xrightarrow{DS} h(n)$. The **step response** $s(n)$ is the output when the input is the unit step $u(n)$: $u(n) \xrightarrow{DS} s(n)$.

⇄ Connections

This is the payoff of the whole chapter. For any *linear time-invariant* (LTI) system, knowing $h(n)$ lets you compute the response to *any* input via the convolution sum

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k)\, h(n-k),$$

which follows directly from writing $x(n)$ in impulse-train form (§ 2.8) and applying linearity (§ 3.5) + time-invariance term by term. This is why the impulse response is called the system's "fingerprint," and it is exactly the derivation carried out step-by-step in § 4.2 of the next chapter.

3.8.2 Worked Example: Accumulator (Three Equivalent Views)

Worked Example: Accumulator impulse and step response

Accumulator-1: $y(n) = \sum_{\ell=-\infty}^{n} x(\ell)$. Impulse response: set $x(\ell) = \delta(\ell)$:

$$h(n) = \sum_{\ell=-\infty}^{n} \delta(\ell) = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases} = u(n).$$

The impulse response of the accumulator is exactly the unit step — once a single impulse of height 1 has entered the running sum (at $\ell = 0$), the running total stays at 1 forever after.

Step response. Set $x(\ell) = u(\ell)$:

$$s(n) = \sum_{\ell=-\infty}^{n} u(\ell) = \sum_{\ell=0}^{n} 1 = (n+1), \quad n \geq 0, \qquad s(n) = 0 \text{ for } n < 0.$$

$$\boxed{s(n) = (n+1)\,u(n)} \quad \text{(a discrete-time ramp).}$$

Check: $s(0) = 1, s(1) = 2, s(2) = 3, s(3) = 4, \ldots$ — summing a constant "1" repeatedly produces linear growth, matching the general LTI fact $s(n) = \sum_{k=-\infty}^{n} h(k)$.
Accumulator-2 (re-indexed): $y(n) = \sum_{k=0}^{n} x(n-k)$ (substituting $\ell = n - k$ recovers Accumulator-1 exactly); its impulse response is again $h(n) = u(n)$.
Accumulator-3 (recursive, with initial condition): $y(n) = y(-1) + \sum_{\ell=0}^{n} x(\ell)$. If the system is initially at rest, $y(-1) = 0$, this reduces to Accumulator-1/2. If $y(-1) \neq 0$, the extra constant is carried forward for all $n \geq 0$ — but a nonzero, input-independent initial condition means the system is not strictly linear/LTI (its zero-input response is nonzero), which is why $h(n)$ and $s(n)$ are always computed assuming zero initial conditions (the system starts at rest).

Exam takeaway

For an LTI system, $s(n) = h(n) * u(n) = \sum_{k=-\infty}^{n} h(k)$: the step response is always the running sum of the impulse response, and conversely $h(n) = s(n) - s(n-1)$. Verify: $s(n) - s(n-1) = (n+1) - n = 1 = h(n)$ for $n \geq 0$. ✓

Bridging to Chapter 4

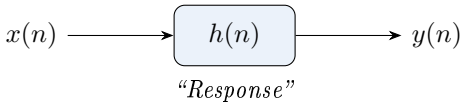
This chapter assembled every ingredient the convolution sum needs: the impulse-train representation (recalled at the top of this chapter), the definition of linearity (§ 3.5), and the impulse response $h(n)$ (§ 3.8). The one ingredient still missing is **time-invariance** — formally defined at the start of Ch. 4 — after which the convolution sum is derived in five short steps.

Chapter 4

LTI Systems and the Convolution Sum

Where we are. Chapter 3 defined the impulse response $h(n)$ and linearity, and recalled the impulse-train representation $x(n) = \sum_k x(k)\delta(n-k)$ from § 2.8. **What this chapter does.** It adds the one missing ingredient, time-invariance, and derives — in five explicit steps mirroring the original handwritten derivation — the single most important formula in introductory DSP: the convolution sum. It then shows the sum is commutative, examines what breaks for genuinely time-varying systems, and specializes to causal systems and causal signals (reusing both causality notions from § 3.6). **Where it leads.** Chapter 5 takes the convolution sum's "fold, shift, multiply, add" recipe and asks what happens if the fold step is removed — producing correlation.

4.1 The Big Picture: System Response via $h(n)$



For a general system, $y(n) = T[x(n)] = f(x)$ — the output is some function of the entire input signal, and in general we need to know the full rule $f(\cdot)$. The question this chapter answers: under what conditions does knowing just *one* signal, $h(n)$, let us compute $y(n)$ for every possible input $x(n)$? The answer: whenever the system is **Linear and Time-Invariant (LTI)**.

Definition: Time-Invariance (TI)

A system is time-invariant if shifting the input in time by any amount k simply shifts the output by the same amount k, with no other change:

$$x(n) \to y(n) \implies x(n-k) \to y(n-k) \quad \text{for every integer } k.$$

Physically: the system behaves the same way today as it will tomorrow — it has no internal clock of its own, and its rule for turning input into output doesn't depend on when the input arrives.

Key Result: Consequence of TI applied to the impulse response

If $\delta(n) \to h(n)$ (definition of impulse response, § 3.8), then by time-invariance alone,

$$\delta(n-k) \longrightarrow h(n-k) \qquad \text{for every integer } k.$$

A shifted impulse produces a shifted (but otherwise identical) impulse response.

4.2 Deriving the Convolution Sum

This is the central derivation of the chapter. We build the response to an arbitrary input $x(n)$ from the response to a single impulse, using the impulse-train representation (§ 2.8) together with linearity (§ 3.5) and time-invariance, one property at a time.

Step-by-step construction

1. **Start from the definition of the impulse response.**
$$\delta(n) \longrightarrow h(n)$$

2. **Apply Time-Invariance (TI).** Shifting the input impulse by k samples shifts the output by k samples:
$$\delta(n-k) \longrightarrow h(n-k)$$

3. **Apply Homogeneity (scaling).** Multiplying the input by a constant multiplies the output by the same constant. Here the "constant" is $x(k)$ — just a fixed number once k is fixed:
$$x(k)\,\delta(n-k) \longrightarrow x(k)\,h(n-k)$$

4. **Apply Superposition (additivity), summed over all k.** Adding up inputs adds up the corresponding outputs — extended from finite sums to the infinite sum over all integers k:
$$\sum_{k=-\infty}^{\infty} x(k)\,\delta(n-k) \longrightarrow \sum_{k=-\infty}^{\infty} x(k)\,h(n-k)$$

5. **Recognize the left-hand side.** From the impulse-train (baseband) representation established in § 2.8,
$$\sum_{k=-\infty}^{\infty} x(k)\,\delta(n-k) = x(n).$$
So the left column is nothing but $x(n)$ itself, decomposed into weighted, shifted impulses — and we already know what output each piece produces.

Key Result: The Convolution Sum

Combining Steps 4 and 5, the output of any LTI system to any input $x(n)$ is

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)\,h(n-k). \tag{4.1}$$

This is the **convolution sum**, written compactly as $y(n) = x(n) * h(n)$. It says: the response of an LTI system to an arbitrary input equals its input convolved with its impulse response. Once $h(n)$ is known, $y(n)$ can be computed for any $x(n)$ — no need to re-derive the system's behaviour from scratch each time.

⇄ Connections

Every arrow in the derivation above is a fact already on record in this book: Step 1 is the definition from § 3.8; Step 3 and Step 4 are homogeneity and additivity from the definition of linearity in § 3.5; Step 5 is the impulse-train identity from § 2.8. Nothing new was assumed except time-invariance, defined at the top of this chapter. This is why the impulse-train representation was flagged, back in Ch. 2, as "the single most-reused fact in the whole book."

4.3 The Equivalent (Commutative) Form

Equation (4.1) can be rewritten by a simple change of summation variable. Let $m = n - k$, so $k = n - m$; as k ranges over all integers, so does m:

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k) \xrightarrow[k=n-m]{m=n-k} y(n) = \sum_{m=-\infty}^{\infty} x(n-m)\,h(m).$$

Renaming the dummy variable $m \to k$:

$$y(n) = \sum_{k=-\infty}^{\infty} h(k)\,x(n-k). \tag{4.2}$$

Key Result: Commutativity of convolution

Equations (4.1) and (4.2) are algebraically identical — they give the same $y(n)$ for every n. This proves

$$x(n) * h(n) = h(n) * x(n),$$

i.e. convolution is commutative: it does not matter whether you think of the system "sliding" over the input or the input "sliding" over the system — the answer is the same. This is sometimes called the *generalized LTI result*, since it shows the input and the impulse response play symmetric, interchangeable roles.

Why both forms are useful

Form (4.1) ($\sum_k x(k)h(n-k)$) is the natural one coming out of the derivation (input decomposed into impulses). Form (4.2) ($\sum_k h(k)x(n-k)$) is often more convenient for hand computation, because $h(k)$ is usually the shorter/simpler sequence to keep fixed while sliding x underneath it (see the worked example in § 4.6).

4.4 A Subtlety: Genuinely Time-Varying Systems

The derivation above leaned on time-invariance in exactly one place (Step 2: $\delta(n-k) \to h(n-k)$). If a system is linear but not time-invariant, Steps 1, 3, 4 still go through, but Step 2 must be replaced by something more general.

Definition: Two-argument impulse response for time-varying systems

For a general linear (not necessarily time-invariant) system, define $h(n,k)$ as the response at time n due to a unit impulse applied at time k. Then, by the same linearity argument as before (Steps 1, 3, 4, without invoking TI),

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)\,h(n,k). \tag{4.3}$$

Key Result: TI $\iff$ $h(n,k)$ depends only on $n-k$

$$h(n,k) = h(n-k) \iff \text{the system is time-invariant.}$$

If $h(n,k) \neq h(n-k)$ — i.e. the response to an impulse depends on *where* in time the impulse was applied, not just on how long ago — the system is time-varying, and the one-argument convolution sum (4.1)/(4.2) does not apply; one must use the full two-argument sum instead.

⇄ Connections

Recall from § 3.5 that $y(n) = n\,x(n)$ is linear but time-varying. Its impulse response to $\delta(n-k)$ is $h(n,k) = n\,\delta(n-k)$, which depends on the absolute time n (not just the lag $n-k$) — consistent with $h(n,k) \neq h(n-k)$ here. This is exactly why time-varying systems need the two-argument form, and exactly why that example was singled out earlier as an "important exam point."

4.5 Special Case: LTIC Systems (Linear, Time-Invariant, Causal)

For the rest of DSP (and for essentially all realizable filters), we further restrict to causal systems. Recall from § 3.6: a system is causal if its output at time n never depends on future input. We now see what causality does to the convolution sum.

4.5.1 Case 1: Causal System, Arbitrary Input

Key Result: Causal system $\Rightarrow h(n) = 0$ for $n < 0$

If the system is causal, its impulse response must satisfy $h(n) = 0$ for $n < 0$. (Intuition: $h(n)$ is the output caused by an impulse at $n = 0$; causality forbids any output before the cause, i.e. for $n < 0$.)

Effect on the convolution sum. Start from (4.1): $y(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k)$. Consider a term where $k > n$. Then the argument of h is $n - k < 0$, so by causality of the system, $h(n-k) = 0$ — that term vanishes automatically:

$$k > n \implies h(n-k) = 0.$$

Hence every term with $k > n$ drops out, and the (formally infinite) sum truncates on its own:

Key Result: Convolution sum for a causal LTI system

$$y(n) = \sum_{k=-\infty}^{n} x(k)\,h(n-k). \tag{4.4}$$

The output at time n depends only on inputs $x(k)$ with $k \leq n$ — i.e. only on present and past input. This is precisely the causality condition on $y(n)$, now derived directly from $h(n) = 0$, $n < 0$, exactly as it should be.

4.5.2 Case 2: Causal System and Causal Input Signal ("LTIC, S&S")

⇄ Connections

Now add the *other* causality notion from § 3.6: a causal *signal* $x(n) = 0$ for $n < 0$. Then in the sum $\sum_{k \leq n} x(k)h(n-k)$, every term with $k < 0$ vanishes too, since $x(k) = 0$ there.

Key Result: Convolution sum, LTIC system with causal (S&S) input

$$y(n) = \sum_{k=0}^{n} x(k)\,h(n-k), \qquad n \geq 0. \tag{4.5}$$

Both limits are now finite: the sum runs only from $k = 0$ to $k = n$. This is the form used in almost all practical filtering/difference-equation problems, where both the system and the signals of interest are causal ("Signal & System," S&S, both causal).

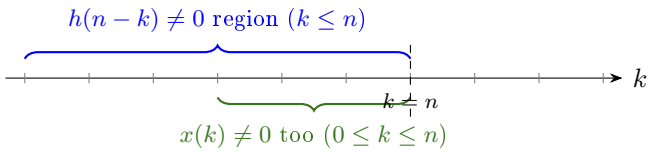


Figure 4.1: Which terms survive in $y(n) = \sum_k x(k)h(n-k)$, shown for $n = 3$. Causality of the system truncates the sum at $k \leq n$; causality of the input additionally truncates it at $k \geq 0$, leaving only $0 \leq k \leq n$.

4.6 Worked Example: Computing a Convolution Sum by Hand

Worked Example: LTIC system with causal input

Let a causal LTI system have impulse response $h(n) = \{\underline{1}, 1, 1\}$ for $n = 0, 1, 2$ (a 3-point moving-sum system, $h(n) = 0$ outside $0 \leq n \leq 2$), and let the causal input be $x(n) = \{\underline{1}, 2, 3\}$ for $n = 0, 1, 2$.
Since both $x(n)$ and $h(n)$ are causal, use (4.5): $y(n) = \sum_{k=0}^{n} x(k)h(n-k)$.
Method: flip, shift, multiply, add. Keep $x(k)$ fixed; use the reversed-and-shifted sequence $h(n-k)$ sliding across it.

$$
\begin{aligned}
y(0) &= x(0)h(0) = (1)(1) = 1 \\
y(1) &= x(0)h(1) + x(1)h(0) = (1)(1) + (2)(1) = 3 \\
y(2) &= x(0)h(2) + x(1)h(1) + x(2)h(0) = (1)(1) + (2)(1) + (3)(1) = 6 \\
y(3) &= x(1)h(2) + x(2)h(1) = (2)(1) + (3)(1) = 5 \quad (x(0)h(3) = 0 \text{ since } h(3) = 0) \\
y(4) &= x(2)h(2) = (3)(1) = 3 \\
y(n) &= 0 \text{ for } n < 0 \text{ or } n > 4.
\end{aligned}
$$

So $y(n) = \{\underline{1}, 3, 6, 5, 3\}$. Note the output has length $3+3-1 = 5$ samples — in general, convolving a length-N sequence with a length-M sequence gives a length-$(N + M - 1)$ result — and this is exactly the moving-sum (unweighted 3-point average $\times 3$) of $x(n)$, consistent with $h(n)$ being a 3-point rectangular window.

Exam tip: the sliding-window / tabular method

For hand computation, write $x(k)$ along one row and a reversed copy of $h(k)$ (i.e. $h(-k)$) below it; to get $y(n)$, shift the reversed row right by n positions, multiply overlapping entries, and sum. This is exactly what "flip h, shift by n, multiply, add" means, and it is the fastest way to avoid sign/index errors under exam time pressure.

4.7 Summary Sheet

Key Result: Chapter 4 formula & concept cheat-sheet

- Impulse response: $\delta(n) \to h(n)$. Time-invariance: $\delta(n-k) \to h(n-k)$.
- Derivation chain: impulse response $\to$ TI $\to$ homogeneity $\to$ superposition $\to$ impulse-train identity $x(n) = \sum_k x(k)\delta(n-k)$.
- Convolution sum (two equivalent, commutative forms):

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k) = \sum_{k=-\infty}^{\infty} h(k)x(n-k) = x(n) * h(n) = h(n) * x(n).$$

- General (possibly time-varying) linear system: $y(n) = \sum_k x(k)h(n,k)$; time-invariant $\iff$ $h(n,k) = h(n-k)$.
- Causal system: $h(n) = 0, n < 0 \Rightarrow y(n) = \sum_{k=-\infty}^{n} x(k)h(n-k)$ (upper limit truncates to n).
- Causal system and causal input (LTIC, S&S): $y(n) = \sum_{k=0}^{n} x(k)h(n-k)$ (both limits finite).
- Hand computation: flip $h(k) \to h(-k)$, shift by n, multiply with $x(k)$, sum $\Rightarrow y(n)$. Output length = (length of x) + (length of h) -1 for finite sequences.

Bridging to Chapter 5

This chapter's central recipe — fold h, shift, multiply, add — is a specific instance of a more general idea: sliding one sequence past another and combining pointwise. Chapter 5 asks the natural next question: what if the two sequences are not "system and input," but two signals we want to *compare*, and what if we skip the folding step? The answer is correlation, and the chapter shows the two operations are related by exactly one sign flip inside the argument of $h(\cdot)$.

Chapter 5

Convolution Properties and Correlation

Where we are. Chapter 4 derived the convolution sum $y(n) = x(n) * h(n) = \sum_k x(k)h(n-k)$ and its "fold–shift–multiply–add" computation recipe. **What this chapter does.** It first restates that recipe explicitly as a four-step algorithm and works two hand examples, then catalogs the algebraic properties of convolution (commutativity, associativity, distributivity, identity element, scalar multiplication, time-shifting, time-reversal). It then introduces **correlation** as the same sliding-sum idea with the fold step removed, defines cross- and auto-correlation, derives the key symmetry relation, and shows correlation is secretly "convolution with one sequence reversed." **Where it leads.** This closes the arc that began with the impulse-train representation in § 2.8: every later use of "slide one sequence past another" in the course (matched filtering, power spectral density estimation via autocorrelation, etc.) is one of the two operations defined precisely here.

5.1 Motivation: Why Convolution at All?

⇄ Connections

This section is a compressed restatement, in one place, of the entire derivation of § 4.2: it is worth having the whole chain in view again before extending it to correlation.

In discrete-time signal processing we are almost always dealing with an LTI system.

Key Idea: LTI systems are fully characterized by their impulse response

If you know how an LTI system responds to a single unit impulse $\delta(n)$ — call this response $h(n)$ — then you can predict the system's output $y(n)$ for any input $x(n)$ whatsoever, without ever touching the system again. The formula that does this is the convolution sum.

The intuition: any discrete signal $x(n)$ decomposes into a weighted sum of shifted impulses, $x(n) = \sum_{k=-\infty}^{\infty} x(k)\delta(n-k)$ — the sifting property of the discrete impulse (§ 2.8). Since the system is linear (superposition holds, § 3.5) and time-invariant (a shifted input produces an identically shifted output, Ch. 4), we can find the response to each impulse piece and add them up: if $\delta(n) \to h(n)$, then by time invariance $\delta(n-k) \to h(n-k)$, and by linearity

$$x(n) = \sum_{k=-\infty}^{\infty} x(k)\delta(n-k) \implies y(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k) = x(n) * h(n).$$

5.2 Convolution

5.2.1 Definition

Definition: Linear convolution

Given two discrete-time sequences $x(n)$ and $h(n)$, their linear convolution is the sequence

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k)\, h(n-k).$$

Here k is a dummy summation variable — it disappears once the sum is evaluated — and n is the free variable indexing the output.

Notice the structure inside the sum: $x(k)$ is left untouched (a function of the dummy variable k), while h appears as $h(n-k)$, i.e. flipped in k and then shifted by n. This gives rise to the graphical/tabular method:

Key Idea: the four-step graphical recipe

Folding $\longrightarrow$ Shifting $\longrightarrow$ Multiplication $\longrightarrow$ Addition

5.2.2 The Four-Step (Fold–Shift–Multiply–Add) Method

Recipe: Computing $y(n)$

Step 1 (Folding/Reflection).
Take $h(k)$ and flip it about the vertical axis ($k = 0$) to get $h(-k)$. This happens once; everything after is repeated for each output index n.

Step 2 (Shifting).
Slide the folded sequence $h(-k)$ right by n (or left if $n < 0$) to obtain $h(n-k)$. Each integer value of n gives one new output sample.

Step 3 (Multiplication).
At the current shift n, multiply $x(k)$ and $h(n-k)$ pointwise, for every k where both are defined (non-zero overlap).

Step 4 (Addition/Summation).
Sum all products from Step 3 across k. This single number is $y(n)$.

Repeat Steps 2–4 for every n for which $x(k)$ and $h(n-k)$ overlap, sliding the folded h across x one step at a time — like a scanning window.

⇄ **Connections**

Step 1 of this recipe is exactly the folding operation defined in § 2.3.2, and Step 2 is exactly the shifting operation from the same section. The abstract algebra of § 2.3.2 was practiced precisely so it could be reused, unchanged, as a computational algorithm here.

Worked Example: Fold–Shift–Multiply–Add by hand

Let $x(n) = \{\underline{1}, 2, 3\}$ and $h(n) = \{\underline{1}, 1\}$ (underline marks $n = 0$). Compute $y(n) = x(n) * h(n)$. Fold $h(k)$ to get $h(-k) = \{1, \underline{1}\}$ (now at positions $k = -1, 0$). Slide across $x(k) = \{1, 2, 3\}$

(positions $k = 0, 1, 2$) and at each shift n, multiply overlapping samples and sum:

$$\begin{aligned} y(0) &= x(0)h(0) = 1 \cdot 1 = 1 \\ y(1) &= x(0)h(1) + x(1)h(0) = 1 \cdot 1 + 2 \cdot 1 = 3 \\ y(2) &= x(1)h(1) + x(2)h(0) = 2 \cdot 1 + 3 \cdot 1 = 5 \\ y(3) &= x(2)h(1) = 3 \cdot 1 = 3 \end{aligned}$$

So $y(n) = \{\underline{1}, 3, 5, 3\}$, of length $3 + 2 - 1 = 4$ samples (as expected: convolving length-N_1 and length-N_2 sequences gives length $N_1 + N_2 - 1$).

To Derive / Work Out

Using the same method, convolve $x(n) = \{1, -1\}$ with itself, i.e. find $x(n) * x(n)$. (Hint: fold one copy, slide it across the other; $y(n)$ will have $2 + 2 - 1 = 3$ samples.)

5.2.3 Correlation

Convolution and correlation look almost identical — both involve summing a product of two sequences — but they answer fundamentally different questions.

Key Idea: Convolution vs. Correlation

Convolution tells you the output of a system given an input and an impulse response. It inherently involves folding one sequence. **Correlation** tells you how similar two sequences are to each other as one is slid past the other. There is *no* folding involved.

Definition: Correlation sum

The correlation of two sequences $x(n)$ and $h(n)$ is

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)\, h(k - n).$$

Compare carefully with the convolution sum: in convolution the shifted/folded term is $h(n - k)$; in correlation it is $h(k - n)$ — the argument order inside $h(\cdot)$ is reversed. This single sign flip is the entire mathematical difference between the two operations, yet it changes their meaning completely: correlation does not fold either sequence before sliding; it just slides h (or slides the reference point) and measures overlap-agreement at each lag n.

Correlation comes in two flavors:

- **Cross-correlation**: comparing two different sequences $x_1(n)$ and $x_2(n)$. Answers "how similar is x_1 to a shifted version of x_2?" Useful in radar/sonar (detecting a known pulse shape buried in a received signal) and in finding time delays between two recordings of the same event.
- **Autocorrelation**: comparing a sequence with a shifted version of itself. Answers "how similar is $x(n)$ to its own past/future?" Useful for detecting periodicity and for estimating power spectra.

5.2.4 Properties of Correlation

The general cross-correlation between $x_1(n)$ and $x_2(n)$ is

$$r_{x_1 x_2}(n) = \sum_{k=-\infty}^{\infty} x_1(k)\, x_2(k - n). \tag{5.1}$$

Key Idea: reading the lag n

$r_{x_1x_2}(n)$ measures the similarity between $x_1(k)$ and $x_2(k)$ delayed by n samples.

- $n > 0$: x_2 has been shifted to the right (delayed) before comparing.
- $n < 0$: x_2 has been shifted to the left (advanced) before comparing.
- The value of n at which $|r_{x_1x_2}(n)|$ peaks tells you the lag at which the two signals line up best — the basis of time-delay estimation.

Key Result: Symmetry relation

$$r_{x_1x_2}(n) = r_{x_2x_1}(-n). \tag{5.2}$$

Derivation. By definition,

$$r_{x_2x_1}(-n) = \sum_k x_2(k)\, x_1(k-(-n)) = \sum_k x_2(k)\, x_1(k+n).$$

Substitute $m = k + n \Rightarrow k = m - n$:

$$= \sum_m x_2(m-n)\, x_1(m) = \sum_m x_1(m)\, x_2(m-n) = r_{x_1x_2}(n).$$

This says: swapping which sequence is "reference" and which is "shifted" is equivalent to reversing the sign of the lag. For autocorrelation ($x_1 = x_2 = x$), this immediately gives the special case

$$r_{xx}(n) = r_{xx}(-n),$$

i.e. the autocorrelation sequence is always even-symmetric about $n = 0$, regardless of whether $x(n)$ itself has any symmetry — one of the most exam-relevant facts about autocorrelation.

To Derive / Work Out

Starting from (5.1), verify the symmetry relation $r_{x_1x_2}(n) = r_{x_2x_1}(-n)$ yourself with the substitution above, then specialize to $x_1 = x_2 = x$ and confirm $r_{xx}(0) \geq |r_{xx}(n)|$ for all n (the autocorrelation is maximum at zero lag — prove this using the Cauchy–Schwarz inequality on the sum).

Key Result: Correlation as convolution with one sequence folded

Since $h(k-n) = h(-(n-k))$, comparing the convolution sum and the correlation sum shows

$$x(n) \star h(n) = x(n) * h(-n),$$

i.e. correlating x with h is the same as convolving x with the time-reversed h. This is extremely useful computationally: any software/hardware that implements convolution (which is most DSP hardware) can compute correlation for free, just by flipping one input first.

⇄ Connections

This last identity is the cleanest possible statement of "correlation = convolution minus folding": flip h first (undoing the fold that convolution itself would apply), and ordinary convolution against the flipped sequence gives correlation. It is the same folding operation from § 5.2.2, now used to *cancel* the fold built into $*$.

5.3 Properties of Convolution

These properties let you manipulate and simplify convolution sums algebraically instead of recomputing the fold-shift-multiply-add procedure from scratch every time.

5.3.1 Properties 1–3: Commutativity, Associativity, Distributivity

Key Result: Properties 1–3

$$x(n) * h(n) = h(n) * x(n) \qquad \text{(Commutative)} \tag{5.3}$$

$$[x(n) * h_1(n)] * h_2(n) = x(n) * [h_1(n) * h_2(n)] \qquad \text{(Associative)} \tag{5.4}$$

$$x(n) * [h_1(n) + h_2(n)] = x(n) * h_1(n) + x(n) * h_2(n) \qquad \text{(Distributive over addition)} \tag{5.5}$$

⇄ Connections

Commutativity (5.3) was already proved in Ch. 4 via the change-of-variable argument that produced the two equivalent forms of the convolution sum, Equations (4.1) and (4.2). It is restated here as the first entry in the complete property list.

Associativity has a physical meaning: cascading two LTI systems h_1 then h_2 is identical to a single LTI system whose impulse response is $h_1(n) * h_2(n)$ — and it doesn't matter in which order you cascade them (by commutativity)!

5.3.2 Property 4: Identity Element

Key Result: Identity element

$$x(n) * \delta(n) = x(n).$$

The unit impulse $\delta(n)$ is the identity element of convolution, exactly the way 1 is the identity for multiplication or 0 is the identity for addition.

Proof. By definition and the sifting property of δ, $x(n) * \delta(n) = \sum_k x(k)\delta(n-k)$. The term $\delta(n-k)$ is 1 only when $k = n$ and 0 otherwise, so the entire (infinite) sum collapses to the single surviving term $x(n)$. □

More generally, convolving with a shifted impulse just shifts the signal: $x(n) * \delta(n-n_0) = x(n-n_0)$.

5.3.3 Property 5: Scalar Multiplication (Homogeneity)

Key Result

$$[a\,x(n)] * h(n) = a\,[x(n) * h(n)].$$

Scaling one input sequence by a constant a scales the entire output by the same constant — really just linearity in disguise: convolution is linear in each of its arguments separately.

Proof. $[a\,x(n)] * h(n) = \sum_k a\,x(k)h(n-k) = a\sum_k x(k)h(n-k) = a\,[x(n) * h(n)]$. □

5.3.4 Property 6: Convolution with a Scaled Impulse

Combining Properties 4 and 5:

$$x(n) * \big[a\,\delta(n)\big] = a\,x(n), \qquad x(n) * \big[a\,\delta(n-n_0)\big] = a\,x(n-n_0).$$

This is exactly what happens physically when a signal reflects off a single perfect reflector: it comes back scaled (by reflection coefficient a) and delayed (by n_0), with no change of shape.

5.3.5 Property 7: Time-Shifting

Key Result

If $x(n) * h(n) = y(n)$, then shifting either input by n_0 shifts the output by the same amount:

$$x(n - n_0) * h(n) = y(n - n_0), \qquad x(n) * h(n - n_0) = y(n - n_0).$$

This is the discrete-time statement of time-invariance baked directly into the convolution operation — delaying the input to an LTI system delays the output by exactly the same amount, with no change in shape. Shifting both inputs simultaneously by n_1 and n_2 shifts the output by $n_1 + n_2$: $x(n - n_1) * h(n - n_2) = y(n - n_1 - n_2)$.

Proof. $x(n - n_0) * h(n) = \sum_k x(k - n_0)h(n - k)$. Substitute $m = k - n_0 \Rightarrow k = m + n_0$: $= \sum_m x(m)h(n - n_0 - m) = \sum_m x(m)h((n - n_0) - m) = y(n - n_0)$. □

5.3.6 Property 8: Time-Reversal

Key Result

If $x(n) * h(n) = y(n)$, then reversing both sequences in time reverses the output:

$$x(-n) * h(-n) = y(-n).$$

This identity requires reversing *both* x and h together. Reversing only one of them does not simply reverse y — it produces a different operation entirely (reversing just h turns convolution into correlation, per § 5.2.3!).

Proof. $x(-n) * h(-n) = \sum_k x(-k)h(-n - k)$. Substitute $m = -k \Rightarrow k = -m$: $= \sum_m x(m)h(-n + m) = \sum_m x(m)h(-(n - m))$. Comparing with $y(-n) = \sum_m x(m)h(-n - m)$ obtained by directly substituting $-n$ into the convolution sum: relabelling the dummy index shows the two sums are identical, confirming $x(-n) * h(-n) = y(-n)$. □

To Derive / Work Out

Given $x(n) * h(n) = y(n)$, is it true that $x(-n) * h(n) = y(-n)$? Explain why or why not, and contrast with Property 8. (Hint: think about which variable is being folded in each case, and revisit the correlation identity from § 5.2.3 for a related but different result.)

5.3.7 Summary Table of Convolution Properties

5.4 Putting It Together: A Worked Comparison

Worked Example: Convolution vs. Correlation on the same pair of sequences

Let $x(n) = \{\underline{2}, 1\}$ and $h(n) = \{\underline{1}, -1\}$.
(a) Convolution $y(n) = x(n) * h(n) = \sum_k x(k)h(n - k)$: fold h: $h(-k) = \{-1, \underline{1}\}$. Slide across x:

$$y(0) = x(0)h(0) = 2(1) = 2$$
$$y(1) = x(0)h(1) + x(1)h(0) = 2(-1) + 1(1) = -1$$
$$y(2) = x(1)h(1) = 1(-1) = -1$$

So $y(n) = \{\underline{2}, -1, -1\}$.

#	Property	Statement
1	Commutative	$x * h = h * x$
2	Associative	$(x * h_1) * h_2 = x * (h_1 * h_2)$
3	Distributive	$x * (h_1 + h_2) = x * h_1 + x * h_2$
4	Identity	$x(n) * \delta(n) = x(n)$
5	Scalar mult.	$[a\,x(n)] * h(n) = a[x(n) * h(n)]$
6	Scaled impulse	$x(n) * [a\,\delta(n - n_0)] = a\,x(n - n_0)$
7	Time shifting	$x(n - n_0) * h(n) = y(n - n_0)$
8	Time reversal	$x(-n) * h(-n) = y(-n)$

Table 5.1: Convolution properties (numbered as in the original course notes).

(b) Correlation $r_{xh}(n) = \sum_k x(k)h(k-n)$ (no folding!):

$$\begin{aligned} r_{xh}(-1) &= x(1)h(0) = 1(1) = 1 \quad \text{(shift } n = -1 : \text{pairs } k - n = k + 1) \\ r_{xh}(0) &= x(0)h(0) + x(1)h(1) = 2(1) + 1(-1) = 1 \\ r_{xh}(1) &= x(0)h(1) = 2(-1) = -2 \end{aligned}$$

So $r_{xh}(n) = \{1, \underline{1}, -2\}$ (underline at $n = 0$).
Notice the two results are different even though the inputs are the same — exactly because correlation skips the folding step that convolution performs.

To Derive / Work Out

Verify part (b) above using the shortcut identity $r_{xh}(n) = x(n) * h(-n)$, and check you get the same three numbers.

5.5 Exam-Ready Summary Sheet

Key Result: Everything on one page

- **Convolution:** $y(n) = \sum_k x(k)h(n-k) = x(n) * h(n)$. Gives the output of an LTI system with impulse response h driven by input x. Involves folding h.
- **Correlation:** $r_{x_1x_2}(n) = \sum_k x_1(k)x_2(k-n)$. Measures similarity between two sequences as a function of lag n. No folding.
- **Link:** $x(n) \star h(n) = x(n) * h(-n)$ — correlation = convolution with one sequence time-reversed first.
- **Symmetry:** $r_{x_1x_2}(n) = r_{x_2x_1}(-n)$; in particular $r_{xx}(n) = r_{xx}(-n)$ (autocorrelation is even).
- **Graphical method (convolution):** Fold → Shift → Multiply → Add.
- **Length rule:** convolving/correlating length-N_1 and length-N_2 finite sequences gives a length-$(N_1 + N_2 - 1)$ result.
- **Convolution properties to know cold:** commutative, associative, distributive, identity element $\delta(n)$, scalar multiplication, time-shifting, time-reversal (needs *both* sequences reversed).

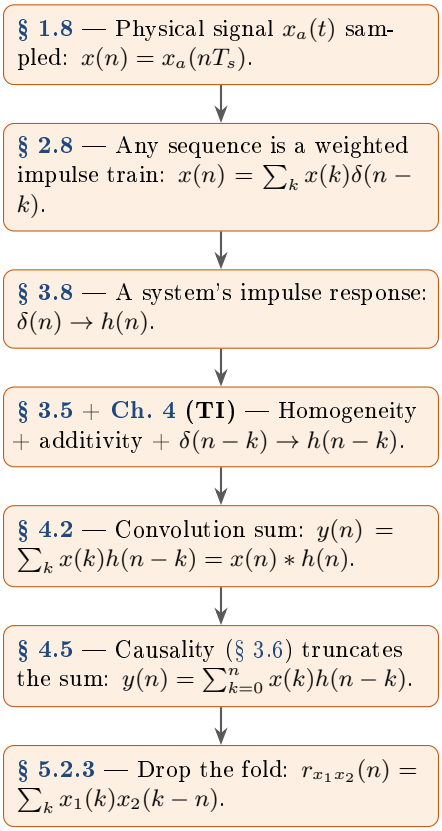
Looking Back

With this chapter, the arc that began in Ch. 1 with a physical signal being sampled and held is complete: that signal became a sequence $x(n)$ (Ch. 2), which was shown to be a weighted train of impulses; that representation, combined with linearity and time-invariance, produced the convolution sum (Ch. 4); and that sum's own machinery — fold, shift, multiply, add — turned out to also describe correlation once the fold is dropped. Section 5.5 collects every key result from all five chapters onto a handful of pages for revision.

Cross-Chapter Summary and Concept Map

This closing chapter is not from any single lecture; it collects, in one place, the results that were built cumulatively across Chs. 1 to 5, and makes explicit which chapter each one first appeared in.

5.6 The Central Thread, in One Derivation Chain



5.7 Master Formula Sheet

Key Result: Chapter 1 — Signals and A/D–D/A

$$\text{Analog} \xrightarrow{\text{Sampling}} \text{Sampled} \xrightarrow{\text{Quantization}} \text{Quantized} \xrightarrow{\text{Encoding}} \text{Digital},$$

$$\Delta = \frac{2A_{\max}}{L}, \quad L = 2^B, \qquad f_c \le \frac{f_s}{2}.$$

Key Result: Chapter 2 — Representation and elementary signals

$$x(n) = \sum_{k=-\infty}^{\infty} x(k)\,\delta(n-k), \qquad \delta(n) = u(n) - u(n-1),$$
$$x_e(n) = \frac{x_m(n) + x_m(-n)}{2}, \qquad x_o(n) = \frac{x_m(n) - x_m(-n)}{2}.$$

Discrete-time sinusoid $\cos(\omega_0 n + \phi)$ periodic $\iff$ $\omega_0/2\pi \in \mathbb{Q}$. Sum of periodics with periods N_1, N_2, N_3: combined period $= \mathrm{lcm}(N_1, N_2, N_3)$.

Key Result: Chapter 3 — Sampling theorem and system properties

$$\omega_0 = \Omega T = 2\pi \frac{f}{f_s}, \qquad f_s > 2f_{\max} \text{ (Nyquist)},$$
$$f_2 = f_1 + k f_s \text{ (aliasing)}.$$

Linearity: $T[a_1x_1 + a_2x_2] = a_1T[x_1] + a_2T[x_2]$. Causality: $x_1(n) = x_2(n), n \le N \Rightarrow y_1(n) = y_2(n), n \le N$. BIBO: bounded $x(n) \Rightarrow$ bounded $y(n)$; for LTI, $\iff \sum_n |h(n)| < \infty$. Accumulator: $h(n) = u(n)$, $s(n) = (n+1)u(n)$.

Key Result: Chapter 4 — Convolution sum

$$y(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k) = \sum_{k=-\infty}^{\infty} h(k)x(n-k) = x(n) * h(n).$$

Causal system: $h(n) = 0, n < 0$. Causal system + causal input: $y(n) = \sum_{k=0}^{n} x(k)h(n-k)$.

Key Result: Chapter 5 — Convolution properties and correlation

Commutative, associative, distributive; identity $x*\delta = x$; scalar mult. $[ax]*h = a[x*h]$; time-shift $x(n-n_0) * h(n) = y(n-n_0)$; time-reversal $x(-n) * h(-n) = y(-n)$.

$$r_{x_1x_2}(n) = \sum_k x_1(k)x_2(k-n), \qquad r_{x_1x_2}(n) = r_{x_2x_1}(-n), \qquad x(n) \star h(n) = x(n) * h(-n).$$

Length rule: length-N_1* length-N_2 $\to$ length-$(N_1 + N_2 - 1)$.

5.8 Where the "To Derive" Exercises Live

Every original "To derive / write up" box has been preserved in place, chapter by chapter, so that working through the book chapter-by-chapter reproduces the full sequence of hand exercises originally assigned across the five lectures: Ch. 1 (the ADC/DAC pipeline and quantization-error derivations), Ch. 2 (impulse-train and periodicity exercises), Ch. 3 (linearity/causality/BIBO worked checks), Ch. 4 (the time-reversal counter-example question), and Ch. 5 (the Cauchy–Schwarz autocorrelation bound and the $x(-n) * h(n)$ question).